JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class AbstractPreferences

java.lang.Object
    java.util.prefs.Preferences
        java.util.prefs.AbstractPreferences

---

```
public abstract class AbstractPreferences
extends Preferences
```

This class provides a skeletal implementation of the `Preferences` class, greatly easing the task of implementing it.

**This class is for `Preferences` implementers only. Normal users of the `Preferences` facility should have no need to consult this documentation. The `Preferences` documentation should suffice.**

Implementors must override the nine abstract service-provider interface (SPI) methods: `getSpi(String)`, `putSpi(String,String)`, `removeSpi(String)`, `childSpi(String)`, `removeNodeSpi()`, `keysSpi()`, `childrenNamesSpi()`, `syncSpi()` and `flushSpi()`. All of the concrete methods specify precisely how they are implemented atop these SPI methods. The implementor may, at his discretion, override one or more of the concrete methods if the default implementation is unsatisfactory for any reason, such as performance.

The SPI methods fall into three groups concerning exception behavior. The `getSpi` method should never throw exceptions, but it doesn't really matter, as any exception thrown by this method will be intercepted by `get(String,String)`, which will return the specified default value to the caller. The `removeNodeSpi, keysSpi, childrenNamesSpi, syncSpi` and `flushSpi` methods are specified to throw `BackingStoreException`, and the implementation is required to throw this checked exception if it is unable to perform the operation. The exception propagates outward, causing the corresponding API method to fail.

The remaining SPI methods `putSpi(String,String)`, `removeSpi(String)` and `childSpi(String)` have more complicated exception behavior. They are not specified to throw `BackingStoreException`, as they can generally obey their contracts even if the backing store is unavailable. This is true because they return no information and their effects are not required to become permanent until a subsequent call to `Preferences.flush()` or `Preferences.sync()`. Generally speaking, these SPI methods should not throw exceptions. In some implementations, there may be circumstances under which these calls cannot even enqueue the requested operation for later processing. Even under these circumstances it is generally better to simply ignore the invocation and return, rather than throwing an exception. Under these circumstances, however, subsequently invoking `flush()` or `sync` would not imply that all previous operations had successfully been made permanent.

There is one circumstance under which `putSpi, removeSpi and childSpi` *should* throw an exception: if the caller lacks sufficient privileges on the underlying operating system to perform the requested operation. This will, for instance, occur on most systems if a non-privileged user attempts to modify system preferences. (The required privileges will vary from implementation to implementation. On some implementations, they are the right to modify the contents of some directory in the file system; on others they are the right to modify contents of some key in a registry.) Under any of these circumstances, it would generally be undesirable to let the program continue executing as if these operations would become permanent at a later time. While implementations are not required to throw an exception under these circumstances, they are encouraged to do so. A `SecurityException` would be appropriate.

Most of the SPI methods require the implementation to read or write information at a preferences node. The implementor should beware of the fact that another VM may have concurrently deleted this node from the backing store. It is the implementation's responsibility to recreate the node if it has been deleted.

Implementation note: In Sun's default `Preferences` implementations, the user's identity is inherited from the underlying operating system and does not change for the lifetime of the virtual machine. It is recognized that server-side `Preferences` implementations may have the user identity change from request to request, implicitly passed to `Preferences` methods via the use of a static `ThreadLocal` instance. Authors of such implementations are *strongly* encouraged to determine the user at the time preferences are accessed (for example by the `get(String,String)` or `put(String,String)` method) rather than permanently associating a user with each `Preferences` instance. The latter behavior conflicts with normal `Preferences` usage and would lead to great confusion.

**Since:**

1.4

**See Also:**

`Preferences`

## *Field Summary*

**Fields**

| Modifier and Type | Field | Description |
|---|---|---|
| `protected final Object` | **`lock`** | An object whose monitor is used to lock this node. |

| | | |
|---|---|---|
| protected boolean | newNode | This field should be true if this node did not exist in the backing store prior to the creation of this object. |

**Fields declared in class Preferences**

MAX_KEY_LENGTH, MAX_NAME_LENGTH, MAX_VALUE_LENGTH

## Constructor Summary

**Constructors**

| Modifier | Constructor | Description |
|---|---|---|
| protected | AbstractPreferences (AbstractPreferences par String name) | Creates a preference node with the specified parent and the specified name relative to its parent. |

## Method Summary

**All Methods** | **Instance Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| String | absolutePath() | Implements the absolutePath method as per the specification in Preferences.absolutePath(). |
| void | addNodeChangeListener (NodeChangeListener ncl) | Registers the specified listener to receive *node change events* for this node. |
| void | addPreferenceChangeListe (PreferenceChangeListene | Registers the specified listener to receive *preference change events* for this preference node. |
| | cachedChildren() | |

| | | |
|---|---|---|
| protected final AbstractPreferences[ | | Returns all known unremoved children of this node. |
| String[] | **childrenNames**() | Implements the children method as per the specification in Preferences.childrenNames(). |
| protected abstract String[] | **childrenNamesSpi**() | Returns the names of the children of this preference node. |
| protected abstract AbstractPreferences | **childSpi**(String name) | Returns the named child of this preference node, creating it if it does not already exist. |
| void | **clear**() | Implements the clear method as per the specification in Preferences.clear(). |
| void | **exportNode**(OutputStream os) | Implements the exportNode method as per the specification in Preferences.exportNode(OutputStream) |
| void | **exportSubtree**(OutputStream os) | Implements the exportSubtree method as per the specification in Preferences.exportSubtree(OutputStre |
| void | **flush**() | Implements the flush method as per the specification in Preferences.flush(). |
| protected abstract void | **flushSpi**() | This method is invoked with this node locked. |
| String | **get**(String key, String def) | Implements the get method as per the specification in Preferences.get(String,String). |

| | | |
|---|---|---|
| boolean | **getBoolean**(String key, boolean def) | Implements the getBoolean method as per the specification in Preferences.getBoolean(String,boolea |
| byte[] | **getByteArray**(String key, byte[] def) | Implements the getByteArray method as per the specification in Preferences.getByteArray(String,byte |
| protected AbstractPreferences | **getChild**(String nodeName) | Returns the named child if it exists, or null if it does not. |
| double | **getDouble**(String key, double def) | Implements the getDouble method as per the specification in Preferences.getDouble(String,double) |
| float | **getFloat**(String key, float def) | Implements the getFloat method as per the specification in Preferences.getFloat(String,float). |
| int | **getInt**(String key, int def) | Implements the getInt method as per the specification in Preferences.getInt(String,int). |
| long | **getLong**(String key, long def) | Implements the getLong method as per the specification in Preferences.getLong(String,long). |
| protected abstract String | **getSpi**(String key) | Return the value associated with the specified key at this preference node, or null if there is no association for this key, or the association cannot be determined at this time. |
| protected boolean | **isRemoved**() | Returns true iff this node (or an ancestor) has been removed with |

| | | |
|---|---|---|
| | | the `removeNode()` method. |
| `boolean` | **`isUserNode`**`()` | Implements the `isUserNode` method as per the specification in `Preferences.isUserNode()`. |
| `String[]` | **`keys`**`()` | Implements the `keys` method as per the specification in `Preferences.keys()`. |
| `protected abstract String[]` | **`keysSpi`**`()` | Returns all of the keys that have an associated value in this preference node. |
| `String` | **`name`**`()` | Implements the `name` method as per the specification in `Preferences.name()`. |
| `Preferences` | **`node`**`(String path)` | Implements the `node` method as per the specification in `Preferences.node(String)`. |
| `boolean` | **`nodeExists`**`(String path)` | Implements the `nodeExists` method as per the specification in `Preferences.nodeExists(String)`. |
| `Preferences` | **`parent`**`()` | Implements the `parent` method as per the specification in `Preferences.parent()`. |
| `void` | **`put`**`(String key, String value)` | Implements the `put` method as per the specification in `Preferences.put(String,String)`. |

| | | |
|---|---|---|
| void | **putBoolean**(String key, boolean value) | Implements the putBoolean method as per the specification in Preferences.putBoolean(String,boolea |
| void | **putByteArray**(String key, byte[] value) | Implements the putByteArray method as per the specification in Preferences.putByteArray(String,byte |
| void | **putDouble**(String key, double value) | Implements the putDouble method as per the specification in Preferences.putDouble(String,double) |
| void | **putFloat**(String key, float value) | Implements the putFloat method as per the specification in Preferences.putFloat(String,float). |
| void | **putInt**(String key, int value) | Implements the putInt method as per the specification in Preferences.putInt(String,int). |
| void | **putLong**(String key, long value) | Implements the putLong method as per the specification in Preferences.putLong(String,long). |
| protected abstract void | **putSpi**(String key, String value) | Put the given key-value association into this preference node. |
| void | **remove**(String key) | Implements the remove(String) method as per the specification in Preferences.remove(String). |
| void | **removeNode**() | Implements the removeNode() method as per the specification in Preferences.removeNode(). |
| void | | |

| | | |
|---|---|---|
| | **removeNodeChangeListener**(NodeChangeListener ncl) | Removes the specified NodeChangeListener, so it no longer receives change events. |
| protected abstract void | **removeNodeSpi**() | Removes this preference node, invalidating it and any preferences that it contains. |
| void | **removePreferenceChangeLi**(PreferenceChangeListene | Removes the specified preference change listener, so it no longer receives preference change events. |
| protected abstract void | **removeSpi**(String key) | Remove the association (if any) for the specified key at this preference node. |
| void | **sync**() | Implements the sync method as per the specification in Preferences.sync(). |
| protected abstract void | **syncSpi**() | This method is invoked with this node locked. |
| String | **toString**() | Returns the absolute path name of this preferences node. |

**Methods declared in class Preferences**

importPreferences, systemNodeForPackage, systemRoot, userNodeForPackage, userRoot

**Methods declared in class Object**

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

## *Field Details*

### newNode

```
protected boolean newNode
```

This field should be `true` if this node did not exist in the backing store prior to the creation of this object. The field is initialized to false, but may be set to true by a subclass constructor (and should not be modified thereafter). This field indicates whether a node change event should be fired when creation is complete.

### lock

```
protected final Object lock
```

An object whose monitor is used to lock this node. This object is used in preference to the node itself to reduce the likelihood of intentional or unintentional denial of service due to a locked node. To avoid deadlock, a node is *never* locked by a thread that holds a lock on a descendant of that node.

## *Constructor Details*

### AbstractPreferences

```
protected AbstractPreferences(AbstractPreferences parent,
                              String name)
```

Creates a preference node with the specified parent and the specified name relative to its parent.

**Parameters:**

`parent` - the parent of this preference node, or null if this is the root.

`name` - the name of this preference node, relative to its parent, or `""` if this is the root.

**Throws:**

`IllegalArgumentException` - if `name` contains a slash (`'/'`), or `parent` is `null` and name isn't `""`.

## *Method Details*

### put

```
public void put(String key,
                String value)
```

Implements the put method as per the specification in Preferences.put(String,String).

This implementation checks that the key and value are legal, obtains this preference node's lock, checks that the node has not been removed, invokes putSpi(String,String), and if there are any preference change listeners, enqueues a notification event for processing by the event dispatch thread.

**Specified by:**

put in class Preferences

**Parameters:**

key - key with which the specified value is to be associated.

value - value to be associated with the specified key.

**Throws:**

NullPointerException - if key or value is null.

IllegalArgumentException - if key.length() exceeds MAX_KEY_LENGTH or if value.length exceeds MAX_VALUE_LENGTH.

IllegalArgumentException - if either key or value contain the null control character, code point U+0000.

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

### get

```
public String get(String key,
                  String def)
```

Implements the get method as per the specification in Preferences.get(String,String).

This implementation first checks to see if key is null throwing a NullPointerException if this is the case. Then it obtains this preference node's lock, checks that the node has not been removed, invokes getSpi(String), and returns the result, unless the getSpi invocation returns null or throws an exception, in which case this invocation returns def.

**Specified by:**

get in class Preferences

**Parameters:**

key - key whose associated value is to be returned.

def - the value to be returned in the event that this preference node has no value associated with key.

**Returns:**

the value associated with key, or def if no value is associated with key.

**Throws:**

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

NullPointerException - if key is null. (A null default *is* permitted.)

IllegalArgumentException - if key contains the null control character, code point U+0000.

## remove

```
public void remove(String key)
```

Implements the remove(String) method as per the specification in Preferences.remove(String).

This implementation obtains this preference node's lock, checks that the node has not been removed, invokes removeSpi(String) and if there are any preference change listeners, enqueues a notification event for processing by the event dispatch thread.

**Specified by:**

remove in class Preferences

**Parameters:**

key - key whose mapping is to be removed from the preference node.

**Throws:**

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

IllegalArgumentException - if key contains the null control character, code point U+0000.

NullPointerException - if key is null.

## clear

```
public void clear()
           throws BackingStoreException
```

Implements the `clear` method as per the specification in `Preferences.clear()`.

This implementation obtains this preference node's lock, invokes `keys()` to obtain an array of keys, and iterates over the array invoking `remove(String)` on each key.

**Specified by:**

`clear` in class `Preferences`

**Throws:**

`BackingStoreException` - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

**See Also:**

`Preferences.removeNode()`

## putInt

```
public void putInt(String key,
                   int value)
```

Implements the `putInt` method as per the specification in `Preferences.putInt(String,int)`.

This implementation translates `value` to a string with `Integer.toString(int)` and invokes `put(String,String)` on the result.

**Specified by:**

`putInt` in class `Preferences`

**Parameters:**

`key` - key with which the string form of value is to be associated.

`value` - value whose string form is to be associated with key.

**Throws:**

`NullPointerException` - if key is `null`.

`IllegalArgumentException` - if `key.length()` exceeds `MAX_KEY_LENGTH`.

`IllegalArgumentException` - if key contains the null control character, code point U+0000.

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

**See Also:**

`Preferences.getInt(String,int)`

## getInt

```
public int getInt(String key,
                  int def)
```

Implements the `getInt` method as per the specification in `Preferences.getInt(String,int)`.

This implementation invokes `get(key, null)`. If the return value is non-null, the implementation attempts to translate it to an `int` with `Integer.parseInt(String)`. If the attempt succeeds, the return value is returned by this method. Otherwise, `def` is returned.

**Specified by:**

`getInt` in class `Preferences`

**Parameters:**

`key` - key whose associated value is to be returned as an int.

`def` - the value to be returned in the event that this preference node has no value associated with `key` or the associated value cannot be interpreted as an int.

**Returns:**

the int value represented by the string associated with `key` in this preference node, or `def` if the associated value does not exist or cannot be interpreted as an int.

**Throws:**

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

`NullPointerException` - if `key` is `null`.

`IllegalArgumentException` - if key contains the null control character, code point U+0000.

**See Also:**

`Preferences.putInt(String,int)`, `Preferences.get(String,String)`

## putLong

```
public void putLong(String key,
                    long value)
```

Implements the `putLong` method as per the specification in `Preferences.putLong(String,long)`.

This implementation translates `value` to a string with `Long.toString(long)` and invokes `put(String,String)` on the result.

**Specified by:**

`putLong` in class `Preferences`

**Parameters:**

`key` - key with which the string form of value is to be associated.

`value` - value whose string form is to be associated with key.

**Throws:**

`NullPointerException` - if key is `null`.

`IllegalArgumentException` - if `key.length()` exceeds `MAX_KEY_LENGTH`.

`IllegalArgumentException` - if key contains the null control character, code point U+0000.

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

**See Also:**

`Preferences.getLong(String,long)`

## getLong

```
public long getLong(String key,
                    long def)
```

Implements the `getLong` method as per the specification in `Preferences.getLong(String,long)`.

This implementation invokes `get(key, null)`. If the return value is non-null, the implementation attempts to translate it to a `long` with `Long.parseLong(String)`. If the attempt succeeds, the return value is returned by this method. Otherwise, `def` is returned.

**Specified by:**

`getLong` in class `Preferences`

**Parameters:**

`key` - key whose associated value is to be returned as a long.

`def` - the value to be returned in the event that this preference node has no value associated with `key` or the associated value cannot be interpreted as a long.

**Returns:**

the long value represented by the string associated with key in this preference node, or def if the associated value does not exist or cannot be interpreted as a long.

**Throws:**

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

NullPointerException - if key is null.

IllegalArgumentException - if key contains the null control character, code point U+0000.

**See Also:**

Preferences.putLong(String,long), Preferences.get(String,String)

## putBoolean

```
public void putBoolean(String key,
                       boolean value)
```

Implements the putBoolean method as per the specification in Preferences.putBoolean(String,boolean).

This implementation translates value to a string with String.valueOf(boolean) and invokes put(String,String) on the result.

**Specified by:**

putBoolean in class Preferences

**Parameters:**

key - key with which the string form of value is to be associated.

value - value whose string form is to be associated with key.

**Throws:**

NullPointerException - if key is null.

IllegalArgumentException - if key.length() exceeds MAX_KEY_LENGTH.

IllegalArgumentException - if key contains the null control character, code point U+0000.

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

**See Also:**

Preferences.getBoolean(String,boolean), Preferences.get(String,String)

## getBoolean

```
public boolean getBoolean(String key,
                          boolean def)
```

Implements the `getBoolean` method as per the specification in `Preferences.getBoolean(String,boolean)`.

This implementation invokes `get(key, null)`. If the return value is non-null, it is compared with `"true"` using `String.equalsIgnoreCase(String)`. If the comparison returns `true`, this invocation returns `true`. Otherwise, the original return value is compared with `"false"`, again using `String.equalsIgnoreCase(String)`. If the comparison returns `true`, this invocation returns `false`. Otherwise, this invocation returns `def`.

**Specified by:**

`getBoolean` in class `Preferences`

**Parameters:**

`key` - key whose associated value is to be returned as a boolean.

`def` - the value to be returned in the event that this preference node has no value associated with `key` or the associated value cannot be interpreted as a boolean.

**Returns:**

the boolean value represented by the string associated with `key` in this preference node, or `def` if the associated value does not exist or cannot be interpreted as a boolean.

**Throws:**

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

`NullPointerException` - if `key` is `null`.

`IllegalArgumentException` - if key contains the null control character, code point U+0000.

**See Also:**

`Preferences.get(String,String)`, `Preferences.putBoolean(String,boolean)`

## putFloat

```
public void putFloat(String key,
                     float value)
```

Implements the `putFloat` method as per the specification in `Preferences.putFloat(String,float)`.

This implementation translates `value` to a string with `Float.toString(float)` and invokes `put(String,String)` on the result.

**Specified by:**

`putFloat` in class `Preferences`

**Parameters:**

`key` - key with which the string form of value is to be associated.

`value` - value whose string form is to be associated with key.

**Throws:**

`NullPointerException` - if key is `null`.

`IllegalArgumentException` - if `key.length()` exceeds `MAX_KEY_LENGTH`.

`IllegalArgumentException` - if key contains the null control character, code point U+0000.

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

**See Also:**

`Preferences.getFloat(String,float)`

## getFloat

```
public float getFloat(String key,
                      float def)
```

Implements the `getFloat` method as per the specification in `Preferences.getFloat(String,float)`.

This implementation invokes `get(key, null)`. If the return value is non-null, the implementation attempts to translate it to an `float` with `Float.parseFloat(String)`. If the attempt succeeds, the return value is returned by this method. Otherwise, `def` is returned.

**Specified by:**

`getFloat` in class `Preferences`

**Parameters:**

`key` - key whose associated value is to be returned as a float.

`def` - the value to be returned in the event that this preference node has no value associated with `key` or the associated value cannot be interpreted as a float.

**Returns:**

the float value represented by the string associated with key in this preference node, or def if the associated value does not exist or cannot be interpreted as a float.

**Throws:**

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

NullPointerException - if key is null.

IllegalArgumentException - if key contains the null control character, code point U+0000.

**See Also:**

Preferences.putFloat(String,float), Preferences.get(String,String)

## putDouble

```
public void putDouble(String key,
                      double value)
```

Implements the putDouble method as per the specification in Preferences.putDouble(String,double).

This implementation translates value to a string with Double.toString(double) and invokes put(String,String) on the result.

**Specified by:**

putDouble in class Preferences

**Parameters:**

key - key with which the string form of value is to be associated.

value - value whose string form is to be associated with key.

**Throws:**

NullPointerException - if key is null.

IllegalArgumentException - if key.length() exceeds MAX_KEY_LENGTH.

IllegalArgumentException - if key contains the null control character, code point U+0000.

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

**See Also:**

Preferences.getDouble(String,double)

## getDouble

```
public double getDouble(String key,
                        double def)
```

Implements the `getDouble` method as per the specification in `Preferences.getDouble(String,double)`.

This implementation invokes `get(key, null)`. If the return value is non-null, the implementation attempts to translate it to an `double` with `Double.parseDouble(String)`. If the attempt succeeds, the return value is returned by this method. Otherwise, `def` is returned.

**Specified by:**

`getDouble` in class `Preferences`

**Parameters:**

`key` - key whose associated value is to be returned as a double.

`def` - the value to be returned in the event that this preference node has no value associated with `key` or the associated value cannot be interpreted as a double.

**Returns:**

the double value represented by the string associated with `key` in this preference node, or `def` if the associated value does not exist or cannot be interpreted as a double.

**Throws:**

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

`NullPointerException` - if `key` is `null`.

`IllegalArgumentException` - if key contains the null control character, code point U+0000.

**See Also:**

`Preferences.putDouble(String,double)`, `Preferences.get(String,String)`

## putByteArray

```
public void putByteArray(String key,
                         byte[] value)
```

Implements the `putByteArray` method as per the specification in `Preferences.putByteArray(String,byte[])`.

**Specified by:**

putByteArray in class Preferences

**Parameters:**

key - key with which the string form of value is to be associated.

value - value whose string form is to be associated with key.

**Throws:**

NullPointerException - if key or value is null.

IllegalArgumentException - if key.length() exceeds MAX_KEY_LENGTH or if value.length exceeds MAX_VALUE_LENGTH*3/4.

IllegalArgumentException - if key contains the null control character, code point U+0000.

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

**See Also:**

Preferences.getByteArray(String,byte[]), Preferences.get(String,String)

## getByteArray

```
public byte[] getByteArray(String key,
                           byte[] def)
```

Implements the getByteArray method as per the specification in Preferences.getByteArray(String,byte[]).

**Specified by:**

getByteArray in class Preferences

**Parameters:**

key - key whose associated value is to be returned as a byte array.

def - the value to be returned in the event that this preference node has no value associated with key or the associated value cannot be interpreted as a byte array.

**Returns:**

the byte array value represented by the string associated with key in this preference node, or def if the associated value does not exist or cannot be interpreted as a byte array.

**Throws:**

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

NullPointerException - if key is null. (A null value for def *is* permitted.)

IllegalArgumentException - if key contains the null control character, code point U+0000.

**See Also:**

Preferences.get(String,String), Preferences.putByteArray(String,byte[])

## keys

```
public String[] keys()
              throws BackingStoreException
```

Implements the keys method as per the specification in Preferences.keys().

This implementation obtains this preference node's lock, checks that the node has not been removed and invokes keysSpi().

**Specified by:**

keys in class Preferences

**Returns:**

an array of the keys that have an associated value in this preference node.

**Throws:**

BackingStoreException - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

IllegalStateException - if this node (or an ancestor) has been removed with the removeNode() method.

## childrenNames

```
public String[] childrenNames()
                       throws BackingStoreException
```

Implements the children method as per the specification in Preferences.childrenNames().

This implementation obtains this preference node's lock, checks that the node has not been removed, constructs a TreeSet initialized to the names of children already cached (the children in this node's "child-cache"), invokes childrenNamesSpi(), and adds all of the returned child-names into the set. The elements of the tree set are dumped into a String array using the toArray method, and this array is returned.

**Specified by:**

childrenNames in class Preferences

**Returns:**

the names of the children of this preference node.

**Throws:**

`BackingStoreException` - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

**See Also:**

`cachedChildren()`

## cachedChildren

```
protected final AbstractPreferences[] cachedChildren()
```

Returns all known unremoved children of this node.

**Returns:**

all known unremoved children of this node.

## parent

```
public Preferences parent()
```

Implements the `parent` method as per the specification in `Preferences.parent()`.

This implementation obtains this preference node's lock, checks that the node has not been removed and returns the parent value that was passed to this node's constructor.

**Specified by:**

`parent` in class `Preferences`

**Returns:**

the parent of this preference node.

**Throws:**

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

## node

```
public Preferences node(String path)
```

Implements the `node` method as per the specification in `Preferences.node(String)`.

This implementation obtains this preference node's lock and checks that the node has not been removed. If `path` is `""`, this node is returned; if `path` is `"/"`, this node's root is returned. If the first character in `path` is not `'/'`, the

implementation breaks `path` into tokens and recursively traverses the path from this node to the named node, "consuming" a name and a slash from `path` at each step of the traversal. At each step, the current node is locked and the node's child-cache is checked for the named node. If it is not found, the name is checked to make sure its length does not exceed `MAX_NAME_LENGTH`. Then the `childSpi(String)` method is invoked, and the result stored in this node's child-cache. If the newly created `Preferences` object's `newNode` field is `true` and there are any node change listeners, a notification event is enqueued for processing by the event dispatch thread.

When there are no more tokens, the last value found in the child-cache or returned by `childSpi` is returned by this method. If during the traversal, two `"/"` tokens occur consecutively, or the final token is `"/"` (rather than a name), an appropriate `IllegalArgumentException` is thrown.

If the first character of `path` is `'/'` (indicating an absolute path name) this preference node's lock is dropped prior to breaking `path` into tokens, and this method recursively traverses the path starting from the root (rather than starting from this node). The traversal is otherwise identical to the one described for relative path names. Dropping the lock on this node prior to commencing the traversal at the root node is essential to avoid the possibility of deadlock, as per the `locking invariant`.

**Specified by:**

`node` in class `Preferences`

**Parameters:**

`path` - the path name of the preference node to return.

**Returns:**

the specified preference node.

**Throws:**

`IllegalArgumentException` - if the path name is invalid (i.e., it contains multiple consecutive slash characters, or ends with a slash character and is more than one character long).

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

**See Also:**

`Preferences.flush()`

## nodeExists

```
public boolean nodeExists(String path)
                   throws BackingStoreException
```

Implements the `nodeExists` method as per the specification in `Preferences.nodeExists(String)`.

This implementation is very similar to `node(String)`, except that `getChild(String)` is used instead of `childSpi(String)`.

**Specified by:**

`nodeExists` in class `Preferences`

**Parameters:**

`path` - the path name of the node whose existence is to be checked.

**Returns:**

true if the specified node exists.

**Throws:**

`BackingStoreException` - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

`IllegalArgumentException` - if the path name is invalid (i.e., it contains multiple consecutive slash characters, or ends with a slash character and is more than one character long).

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method and `pathname` is not the empty string ("").

## removeNode

```
public void removeNode()
                throws BackingStoreException
```

Implements the `removeNode()` method as per the specification in `Preferences.removeNode()`.

This implementation checks to see that this node is the root; if so, it throws an appropriate exception. Then, it locks this node's parent, and calls a recursive helper method that traverses the subtree rooted at this node. The recursive method locks the node on which it was called, checks that it has not already been removed, and then ensures that all of its children are cached: The `childrenNamesSpi()` method is invoked and each returned child name is checked for containment in the child-cache. If a child is not already cached, the `childSpi(String)` method is invoked to create a `Preferences` instance for it, and this instance is put into the child-cache. Then the helper method calls itself recursively on each node contained in its child-cache. Next, it invokes `removeNodeSpi()`, marks itself as removed, and removes itself from its parent's child-cache. Finally, if there are any node change listeners, it enqueues a notification event for processing by the event dispatch thread.

Note that the helper method is always invoked with all ancestors up to the "closest non-removed ancestor" locked.

**Specified by:**

`removeNode` in class `Preferences`

**Throws:**

`IllegalStateException` - if this node (or an ancestor) has already been removed with the `removeNode()` method.

`UnsupportedOperationException` - if this method is invoked on the root node.

`BackingStoreException` - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

**See Also:**

`Preferences.flush()`

## name

```
public String name()
```

Implements the `name` method as per the specification in `Preferences.name()`.

This implementation merely returns the name that was passed to this node's constructor.

**Specified by:**

`name` in class `Preferences`

**Returns:**

this preference node's name, relative to its parent.

## absolutePath

```
public String absolutePath()
```

Implements the `absolutePath` method as per the specification in `Preferences.absolutePath()`.

This implementation merely returns the absolute path name that was computed at the time that this node was constructed (based on the name that was passed to this node's constructor, and the names that were passed to this node's ancestors' constructors).

**Specified by:**

`absolutePath` in class `Preferences`

**Returns:**

this preference node's absolute path name.

## isUserNode

```
public boolean isUserNode()
```

Implements the `isUserNode` method as per the specification in `Preferences.isUserNode()`.

This implementation compares this node's root node (which is stored in a private field) with the value returned by `Preferences.userRoot()`. If the two object references are identical, this method returns true.

**Specified by:**

`isUserNode` in class `Preferences`

**Returns:**

`true` if this preference node is in the user preference tree, `false` if it's in the system preference tree.

## addPreferenceChangeListener

```
public void addPreferenceChangeListener(PreferenceChangeListener pcl)
```

**Description copied from class: `Preferences`**

Registers the specified listener to receive *preference change events* for this preference node. A preference change event is generated when a preference is added to this node, removed from this node, or when the value associated with a preference is changed. (Preference change events are *not* generated by the `Preferences.removeNode()` method, which generates a *node change event*. Preference change events *are* generated by the `clear` method.)

Events are only guaranteed for changes made within the same JVM as the registered listener, though some implementations may generate events for changes made outside this JVM. Events may be generated before the changes have been made persistent. Events are not generated when preferences are modified in descendants of this node; a caller desiring such events must register with each descendant.

**Specified by:**

`addPreferenceChangeListener` in class `Preferences`

**Parameters:**

`pcl` - The preference change listener to add.

**See Also:**

`Preferences.removePreferenceChangeListener(PreferenceChangeListener)`, `Preferences.addNodeChangeListener(NodeChangeListener)`

## removePreferenceChangeListener

```
public void removePreferenceChangeListener
(PreferenceChangeListener pcl)
```

**Description copied from class: Preferences**

Removes the specified preference change listener, so it no longer receives preference change events.

**Specified by:**

removePreferenceChangeListener in class Preferences

**Parameters:**

pcl - The preference change listener to remove.

**See Also:**

Preferences.addPreferenceChangeListener(PreferenceChangeListener)

## addNodeChangeListener

```
public void addNodeChangeListener(NodeChangeListener ncl)
```

**Description copied from class: Preferences**

Registers the specified listener to receive *node change events* for this node. A node change event is generated when a child node is added to or removed from this node. (A single Preferences.removeNode() invocation results in multiple *node change events*, one for every node in the subtree rooted at the removed node.)

Events are only guaranteed for changes made within the same JVM as the registered listener, though some implementations may generate events for changes made outside this JVM. Events may be generated before the changes have become permanent. Events are not generated when indirect descendants of this node are added or removed; a caller desiring such events must register with each descendant.

Few guarantees can be made regarding node creation. Because nodes are created implicitly upon access, it may not be feasible for an implementation to determine whether a child node existed in the backing store prior to access (for example, because the backing store is unreachable or cached information is out of date). Under these circumstances, implementations are neither required to generate node change events nor prohibited from doing so.

**Specified by:**

addNodeChangeListener in class Preferences

**Parameters:**

ncl - The NodeChangeListener to add.

**See Also:**

`Preferences.removeNodeChangeListener(NodeChangeListener)`, `Preferences.addPreferenceChangeListener(PreferenceChangeListener)`

## removeNodeChangeListener

```
public void removeNodeChangeListener(NodeChangeListener ncl)
```

### Description copied from class: Preferences

Removes the specified `NodeChangeListener`, so it no longer receives change events.

**Specified by:**

`removeNodeChangeListener` in class `Preferences`

**Parameters:**

`ncl` - The `NodeChangeListener` to remove.

**See Also:**

`Preferences.addNodeChangeListener(NodeChangeListener)`

## putSpi

```
protected abstract void putSpi(String key,
                               String value)
```

Put the given key-value association into this preference node. It is guaranteed that `key` and `value` are non-null and of legal length. Also, it is guaranteed that this node has not been removed. (The implementor needn't check for any of these things.)

This method is invoked with the lock on this node held.

**Parameters:**

`key` - the key

`value` - the value

## getSpi

```
protected abstract String getSpi(String key)
```

Return the value associated with the specified key at this preference node, or `null` if there is no association for this key, or the association cannot be determined at this time. It is guaranteed that `key` is non-null. Also, it is guaranteed that this node has not been removed. (The implementor needn't check for either of these things.)

Generally speaking, this method should not throw an exception under any circumstances. If, however, if it does throw an exception, the exception will be intercepted and treated as a `null` return value.

This method is invoked with the lock on this node held.

**Parameters:**

key - the key

**Returns:**

the value associated with the specified key at this preference node, or `null` if there is no association for this key, or the association cannot be determined at this time.

## removeSpi

```
protected abstract void removeSpi(String key)
```

Remove the association (if any) for the specified key at this preference node. It is guaranteed that key is non-null. Also, it is guaranteed that this node has not
been removed. (The implementor needn't check for either of these things.)

This method is invoked with the lock on this node held.

**Parameters:**

key - the key

## removeNodeSpi

```
protected abstract void removeNodeSpi()
                                throws BackingStoreException
```

Removes this preference node, invalidating it and any preferences that it contains. The named child will have no descendants at the time this invocation is made (i.e., the `Preferences.removeNode()` method invokes this method repeatedly in a bottom-up fashion, removing each of a node's descendants before removing the node itself).

This method is invoked with the lock held on this node and its parent (and all ancestors that are being removed as a result of a single invocation to `Preferences.removeNode()`).

The removal of a node needn't become persistent until the `flush` method is invoked on this node (or an ancestor).

If this node throws a `BackingStoreException`, the exception will propagate out beyond the enclosing `removeNode()` invocation.

**Throws:**

BackingStoreException - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

## keysSpi

```
protected abstract String[] keysSpi()
                            throws BackingStoreException
```

Returns all of the keys that have an associated value in this preference node. (The returned array will be of size zero if this node has no preferences.) It is guaranteed that this node has not been removed.

This method is invoked with the lock on this node held.

If this node throws a `BackingStoreException`, the exception will propagate out beyond the enclosing `keys()` invocation.

**Returns:**

an array of the keys that have an associated value in this preference node.

**Throws:**

BackingStoreException - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

## childrenNamesSpi

```
protected abstract String[] childrenNamesSpi()
                                      throws BackingStoreException
```

Returns the names of the children of this preference node. (The returned array will be of size zero if this node has no children.) This method need not return the names of any nodes already cached, but may do so without harm.

This method is invoked with the lock on this node held.

If this node throws a `BackingStoreException`, the exception will propagate out beyond the enclosing `childrenNames()` invocation.

**Returns:**

an array containing the names of the children of this preference node.

**Throws:**

BackingStoreException - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

## getChild

```
protected AbstractPreferences getChild(String nodeName)
                                throws BackingStoreException
```

Returns the named child if it exists, or `null` if it does not. It is guaranteed that `nodeName` is non-null, non-empty, does not contain the slash character ('/'), and is no longer than `Preferences.MAX_NAME_LENGTH` characters. Also, it is guaranteed that this node has not been removed. (The implementor needn't check for any of these things if he chooses to override this method.)

Finally, it is guaranteed that the named node has not been returned by a previous invocation of this method or `childSpi(String)` after the last time that it was removed. In other words, a cached value will always be used in preference to invoking this method. (The implementor needn't maintain his own cache of previously returned children if he chooses to override this method.)

This implementation obtains this preference node's lock, invokes `childrenNames()` to get an array of the names of this node's children, and iterates over the array comparing the name of each child with the specified node name. If a child node has the correct name, the `childSpi(String)` method is invoked and the resulting node is returned. If the iteration completes without finding the specified name, `null` is returned.

**Parameters:**

`nodeName` - name of the child to be searched for.

**Returns:**

the named child if it exists, or null if it does not.

**Throws:**

`BackingStoreException` - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

## childSpi

```
protected abstract AbstractPreferences childSpi(String name)
```

Returns the named child of this preference node, creating it if it does not already exist. It is guaranteed that `name` is non-null, non-empty, does not contain the slash character ('/'), and is no longer than `Preferences.MAX_NAME_LENGTH` characters. Also, it is guaranteed that this node has not been removed. (The implementor needn't check for any of these things.)

Finally, it is guaranteed that the named node has not been returned by a previous invocation of this method or `getChild(String)` after the last time

that it was removed. In other words, a cached value will always be used in preference to invoking this method. Subclasses need not maintain their own cache of previously returned children.

The implementer must ensure that the returned node has not been removed. If a like-named child of this node was previously removed, the implementer must return a newly constructed `AbstractPreferences` node; once removed, an `AbstractPreferences` node cannot be "resuscitated."

If this method causes a node to be created, this node is not guaranteed to be persistent until the `flush` method is invoked on this node or one of its ancestors (or descendants).

This method is invoked with the lock on this node held.

**Parameters:**

`name` - The name of the child node to return, relative to this preference node.

**Returns:**

The named child node.

## toString

```
public String toString()
```

Returns the absolute path name of this preferences node.

**Specified by:**

`toString` in class `Preferences`

**Returns:**

a string representation of the object

## sync

```
public void sync()
          throws BackingStoreException
```

Implements the `sync` method as per the specification in `Preferences.sync()`.

This implementation calls a recursive helper method that locks this node, invokes syncSpi() on it, unlocks this node, and recursively invokes this method on each "cached child." A cached child is a child of this node that has been created in this VM and not subsequently removed. In effect, this method does a depth first traversal of the "cached subtree" rooted at this node, calling syncSpi() on each node in the subTree while only that node is locked. Note that syncSpi() is invoked top-down.

**Specified by:**

`sync` in class `Preferences`

**Throws:**

`BackingStoreException` - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

`IllegalStateException` - if this node (or an ancestor) has been removed with the `removeNode()` method.

**See Also:**

`flush()`

## syncSpi

```
protected abstract void syncSpi()
                        throws BackingStoreException
```

This method is invoked with this node locked. The contract of this method is to synchronize any cached preferences stored at this node with any stored in the backing store. (It is perfectly possible that this node does not exist on the backing store, either because it has been deleted by another VM, or because it has not yet been created.) Note that this method should *not* synchronize the preferences in any subnodes of this node. If the backing store naturally syncs an entire subtree at once, the implementer is encouraged to override sync(), rather than merely overriding this method.

If this node throws a `BackingStoreException`, the exception will propagate out beyond the enclosing `sync()` invocation.

**Throws:**

`BackingStoreException` - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

## flush

```
public void flush()
           throws BackingStoreException
```

Implements the `flush` method as per the specification in `Preferences.flush()`.

This implementation calls a recursive helper method that locks this node, invokes flushSpi() on it, unlocks this node, and recursively invokes this method on each "cached child." A cached child is a child of this node that has been created in this VM and not subsequently removed. In effect, this method does a depth first traversal of the "cached subtree" rooted at this node, calling

flushSpi() on each node in the subTree while only that node is locked. Note that flushSpi() is invoked top-down.

If this method is invoked on a node that has been removed with the `removeNode()` method, flushSpi() is invoked on this node, but not on others.

**Specified by:**

`flush` in class `Preferences`

**Throws:**

`BackingStoreException` - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

**See Also:**

`flush()`

## flushSpi

```
protected abstract void flushSpi()
                        throws BackingStoreException
```

This method is invoked with this node locked. The contract of this method is to force any cached changes in the contents of this preference node to the backing store, guaranteeing their persistence. (It is perfectly possible that this node does not exist on the backing store, either because it has been deleted by another VM, or because it has not yet been created.) Note that this method should *not* flush the preferences in any subnodes of this node. If the backing store naturally flushes an entire subtree at once, the implementer is encouraged to override flush(), rather than merely overriding this method.

If this node throws a `BackingStoreException`, the exception will propagate out beyond the enclosing `flush()` invocation.

**Throws:**

`BackingStoreException` - if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

## isRemoved

```
protected boolean isRemoved()
```

Returns `true` iff this node (or an ancestor) has been removed with the `removeNode()` method. This method locks this node prior to returning the contents of the private field used to track this state.

**Returns:**

`true` iff this node (or an ancestor) has been removed with the `removeNode()` method.

## exportNode

```
public void exportNode(OutputStream os)
                throws IOException,
                       BackingStoreException
```

Implements the `exportNode` method as per the specification in `Preferences.exportNode(OutputStream)`.

**Specified by:**

`exportNode` in class `Preferences`

**Parameters:**

`os` - the output stream on which to emit the XML document.

**Throws:**

`IOException` - if writing to the specified output stream results in an `IOException`.

`BackingStoreException` - if preference data cannot be read from backing store.

**See Also:**

`Preferences.importPreferences(InputStream)`

## exportSubtree

```
public void exportSubtree(OutputStream os)
                  throws IOException,
                         BackingStoreException
```

Implements the `exportSubtree` method as per the specification in `Preferences.exportSubtree(OutputStream)`.

**Specified by:**

`exportSubtree` in class `Preferences`

**Parameters:**

`os` - the output stream on which to emit the XML document.

**Throws:**

`IOException` - if writing to the specified output stream results in an `IOException`.

`BackingStoreException` - if preference data cannot be read from backing store.

**See Also:**

`Preferences.importPreferences(InputStream)`, `Preferences.exportNode(OutputStream)`

Report a bug or suggest an enhancement

For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.